

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ
Кафедра програмної інженерії

ЗВІТ
з лабораторної роботи № 4
«Рефакторинг коду та проведення Code Review
за індустріальними стандартами»
з дисципліни
«Основи програмної інженерії»

Виконав: студент групи ПЗПІ-25-4
Купрін Матвій Володимирович
Перевірив:
В. О. Гребенюк

Харків 2026

4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

4.2 Хід роботи

4.2.1 Посилання на код модуля та тести з ЛР 03

<https://github.com/mk1234717/MenuQR/tree/KuprinMatvii/Unit%20Testing>

4.2.2 Встановлення SonarQube for IDE (SonarLint) у VS Code та фіксація початкової кількості issues

Було встановлено SonarQube for IDE у VS Code, це показано на рисунку 4.1.

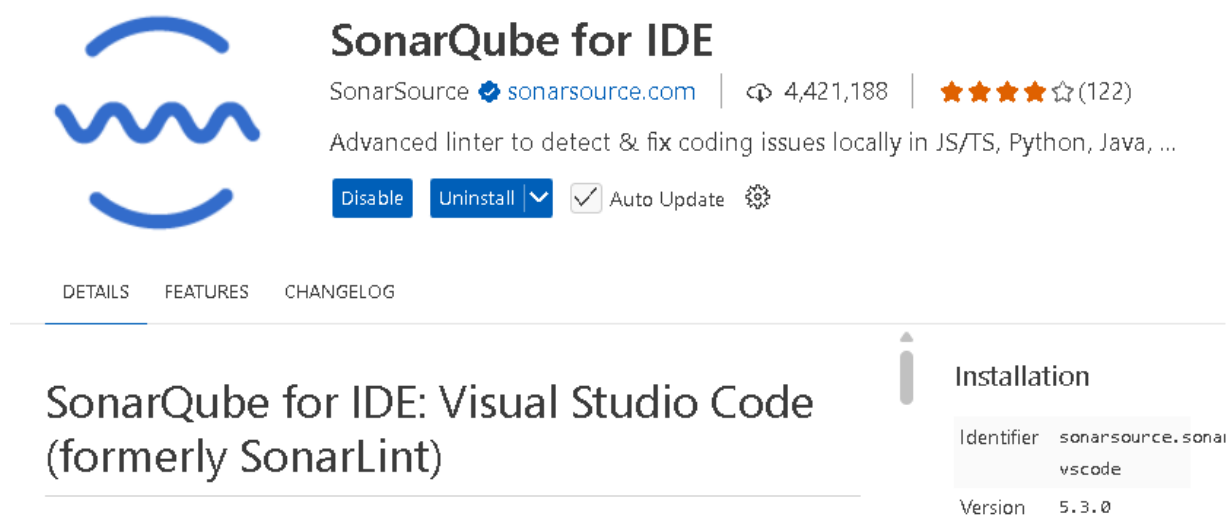


Рисунок 4.1 – Встановлено розширення SonarQube for IDE

Початкова кількість проблем = 0, це показано на рисунку 4.2.

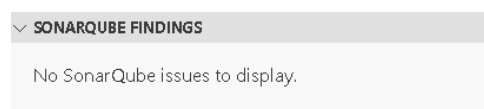


Рисунок 4.2 – Знаходження проблем

4.2.3 Запуск літера Ruff та фіксація кількості попереджень

Було встановлено Ruff у VS Code, це показано на рисунку 4.3.

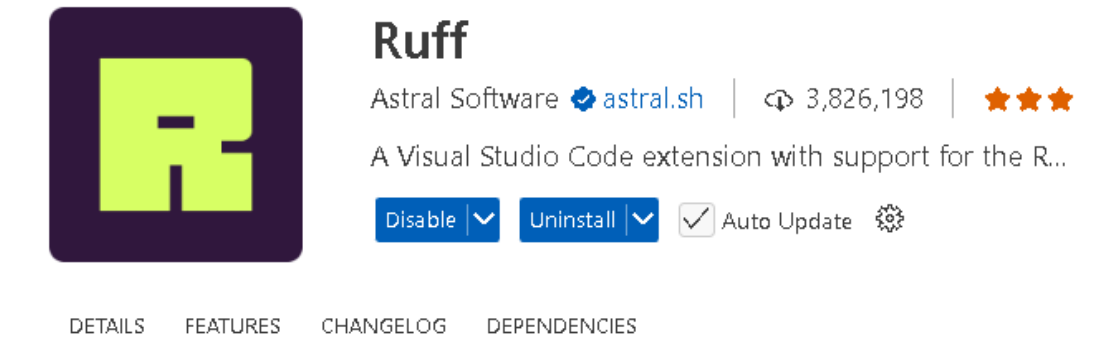


Рисунок 4.3 – Встановлено розширення Ruff

Початкова кількість попереджень = 0, це показано на рисунку 4.4.

```
PS D:\Домашнє завдання\Інформатика\ООП\MenuQR> python -m ruff check "Unit Testing\menu_item.py"
All checks passed!
```

Рисунок 4.4 – Знаходження попереджень

4.2.4 Обчислення метрик цикломатичної складності та індексу супроводжуваності

Для обчислення метрик цикломатичної складності та індексу супроводжуваності було використано radon, результати: цикломатична складність = 4.9, індекс супроводжуваності = 61 – дані підтверджено на рисунку 4.5.

```
PS D:\Домашнє завдання\Інформатика\ООП\MenuQR> python -m radon cc "Unit Testing\menu_item.py" -a
Unit Testing\menu_item.py
M 100:4 MenuItem.validate_for_publish - C
M 24:4 MenuItem.__init__ - B
C 18:0 MenuItem - B
M 48:4 MenuItem.calculate_final_price - B
M 143:4 MenuItem.set_special_discount - A
C 12:0 ValidationResult - A
C 5:0 SeasonalDiscount - A
M 14:4 ValidationResult.__init__ - A
M 80:4 MenuItem.calculate_price_with_vat - A
M 7:4 SeasonalDiscount.__init__ - A
M 140:4 MenuItem.set_availability - A

11 blocks (classes, functions, methods) analyzed.
Average complexity: A (4.909090909090909)
PS D:\Домашнє завдання\Інформатика\ООП\MenuQR> python -m radon mi "Unit Testing\menu_item.py" -s
Unit Testing\menu_item.py - A (60.87)
```

Рисунок 4.5 - Обчислення метрик цикломатичної складності та індексу супроводжуваності

4.2.5 Отримання доступу до репозиторію одnogрупника

Доступ отримано шляхом запрошення collaborator. На рисунку 4.5 показано список collaboratorів, що підтверджує наявність прав на читання та коментування коду.

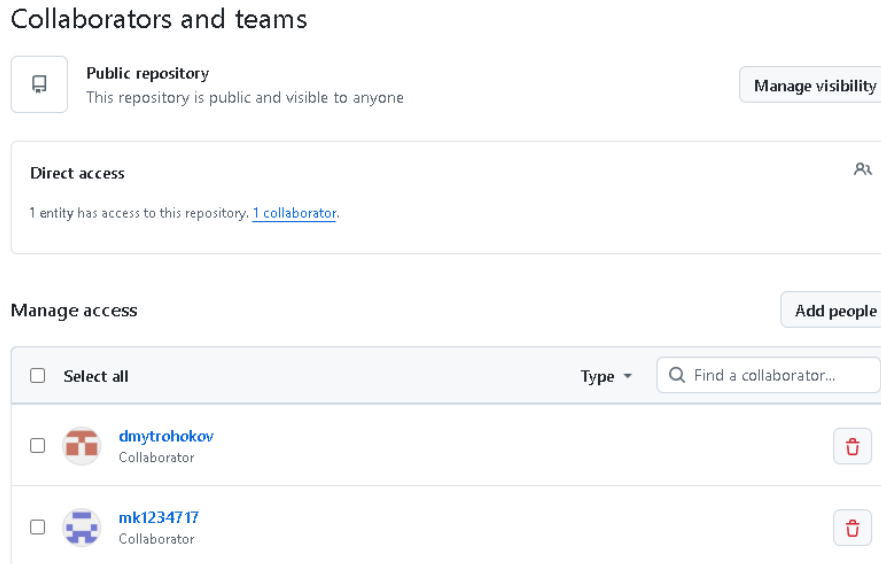


Рисунок 4.6 – Список collaboratorів

4.2.6 Створення нової гілки та відкриття Pull Request

Було створено нову гілку review-KuprinMatvii, рисунок 4.6 підтверджує це.

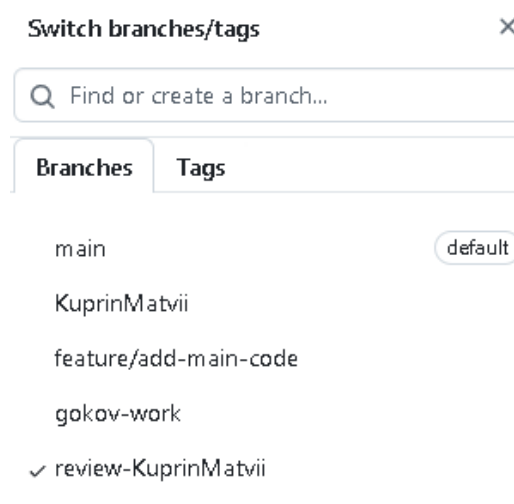


Рисунок 4.7 – Гілки репозиторія

Було відкрито Pull Request, рисунок 4.8 підтверджує це.

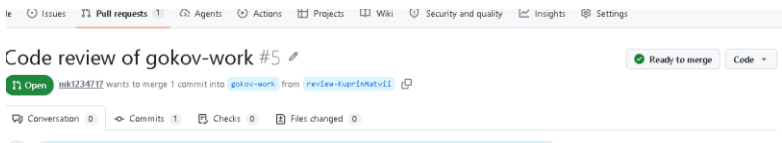


Рисунок 4.8 – Вікно Pull Request

4.2.7 Проведення Code Review: виявлення 5 проблем якості коду. Документування кожної проблеми у вигляді inline-коментаря в PR

Було виявлено 5 проблем якості коду, вони зазначені у таблиці 4.1

Таблиця 4.1 – Code Review

№	Рядок коду	Проблема	Категорія code smell	Конкретна рекомендація щодо виправлення
1	19	Межа кількості однієї страви (20) використана безпосередньо у коді.	Магічне число	Винести у константу класу або модуля: MAX_QUANTITY_PER_ITEM = 20
2	29	Межі номера столика (1 та 50) задані як літерали.	Магічне число	Оголосити константи: MIN_TABLE_NUMBER = 1 MAX_TABLE_NUMBER = 50
3	46	Перевірка кількості страви (не більше 20) повторюється в OrderItem.__init__ (рядок 19) та тут. Зміна ліміту потребує правки у двох місцях.	Дублювання коду (порушення DRY)	Винести логіку в окремий статичний метод _validate_quantity(quantity) або використовувати спільну константу.
4	72	Максимальний відсоток чайових (50) заданий як літерал.	Магічне число	Константа: MAX_TIP_PERCENT = 50
5	78	Розрахунки ведуться в Decimal, але результат перетворюється у float. Це нівелює переваги точного десяткового типу і може спричинити помилки округлення.	Надмірна складність / втрата точності	Повертати Decimal (або залишати float тільки на межі системи). Якщо потрібен float – виконувати перетворення в останній момент, але краще змінити сигнатуру методу на -> Decimal і використовувати quantize.

Всі ці зауваження було задокументовано у вигляді inline-коментарів у PR, посилання на PR: <https://github.com/mk1234717/MenuQR/pull/7>.

4.2.8 Рефакторинг коду

Було отримано результати Code Review від одногрупника:

<https://github.com/mk1234717/MenuQR/pull/8>

Операція №1. Заміна магічного числа константою

Проблема: У конструкторі MenuItem число 1000000 використане безпосередньо для перевірки максимальної ціни. Це ускладнює розуміння та підтримку коду.

БУЛО:

```
def __init__(self, name: str, price: float, image_url: str = None,
            is_available: bool = True, special_discount_percent: Optional[float] = None):
```

```
# ...
```

```
if price > 1_000_000:
```

```
    raise ValueError("Ціна перевищує максимально допустиму (1 000 000).")
```

```
# ...
```

СТАЛО:

```
class MenuItem:
```

```
    MAX_PRICE = 1_000_000 # константа класу
```

```
def __init__(self, name: str, price: float, image_url: str = None,
```

```
            is_available: bool = True, special_discount_percent: Optional[float] = None):
```

```
# ...
```

```
if price > self.MAX_PRICE:
```

```
    raise ValueError(f"Ціна перевищує максимально допустиму ({self.MAX_PRICE}).")
```

```
# ...
```

ЧОМУ:

- Магічне число отримало змістовне ім'я MAX_PRICE, що пояснює бізнес-обмеження.
- Зміна ліміту потребує правки лише в одному місці (константа), а не пошуку числа в коді.
- Відповідає принципу DRY та покращує читабельність.

Було проведено регресійне тестування (тести з ЛР 03, посилання на них: https://github.com/mk1234717/MenuQR/blob/KuprinMatvii/Unit%20Testing/test_menu_item.py). Регресійне тестування пройдено, рисунок 4.9 підтверджує це.

```
✓ Tests passed: 12 of 12 tests – 14 ms

===== test session starts =====
collecting ... collected 12 items

test_menu_item.py::test_calculate_final_price_available_no_discount PASSED [ 8%]
test_menu_item.py::test_calculate_final_price_unavailable PASSED [ 16%]
test_menu_item.py::test_calculate_final_price_special_discount_25 PASSED [ 25%]
test_menu_item.py::test_calculate_final_price_special_discount_boundaries PASSED [ 33%]
test_menu_item.py::test_calculate_final_price_multiple_seasonal_discounts PASSED [ 41%]
test_menu_item.py::test_calculate_price_with_vat_standard PASSED [ 50%]
test_menu_item.py::test_calculate_price_with_vat_reduced PASSED [ 58%]
test_menu_item.py::test_calculate_price_with_vat_invalid_rate PASSED [ 66%]
test_menu_item.py::test_calculate_price_with_vat_price_boundaries PASSED [ 75%]
test_menu_item.py::test_validate_for_publish_valid PASSED [ 83%]
test_menu_item.py::test_validate_for_publish_name_boundaries PASSED [ 91%]
test_menu_item.py::test_validate_for_publish_price_boundaries PASSED [100%]

===== 12 passed in 0.18s =====
```

Рисунок 4.9 – Проходження тестів з третьої лабораторної

Операція №2. Розбиття довгого методу (Extract Method)

Проблема: Метод `validate_for_publish` містив ~35 рядків, перевіряв чотири різні аспекти (назва, ціна, знижка, URL), порушуючи SRP.

БУЛО: `def validate_for_publish(self) -> ValidationResult:`

```
errors = []
```

```
# перевірка назви
```

```
if not self.name or len(self.name.strip()) < 2:
```

```
    errors.append("Назва повинна містити хоча б 2 символи.")
```

```
elif len(self.name) > 100:
```

```
    errors.append("Назва не може перевищувати 100 символів.")
```

```
# перевірка ціни
```

```
if self.price < 0:
```

```

        errors.append("Ціна не може бути від'ємною.")
    elif self.price == 0:
        errors.append("Ціна не може дорівнювати нулю...")
    elif self.price > 10000:
        errors.append("Ціна занадто висока (> 10000).")
    # перевірка знижки
    if self.special_discount_percent is not None:
        if self.special_discount_percent < 0 or self.special_discount_percent > 100:
            errors.append("Знижка має бути від 0% до 100%.")
        elif self.special_discount_percent == 100 and self.price > 0:
            errors.append("Страва не може бути повністю безкоштовною...")
    # перевірка URL
    if self.image_url:
        import re
        url_pattern = re.compile(r'^https?://\S+$')
        if not url_pattern.match(self.image_url):
            errors.append("Посилання на зображення має бути коректним URL...")
    return ValidationResult(is_valid=len(errors)==0, errors=errors)

СТАЛО: def validate_for_publish(self) -> ValidationResult:
    errors = []
    self._validate_name(errors)
    self._validate_price(errors)
    self._validate_discount(errors)
    self._validate_image_url(errors)
    return ValidationResult(is_valid=len(errors) == 0, errors=errors)

def _validate_name(self, errors: List[str]) -> None:
    if not self.name or len(self.name.strip()) < 2:

```

```
errors.append("Назва повинна містити хоча б 2 символи.")

elif len(self.name) > 100:

    errors.append("Назва не може перевищувати 100 символів.")

def _validate_price(self, errors: List[str]) -> None:

    if self.price < 0:

        errors.append("Ціна не може бути від'ємною.")

    elif self.price == 0:

        errors.append("Ціна не може дорівнювати нулю (безкоштовні страви не підтримуються).")

    elif self.price > 10000:

        errors.append("Ціна занадто висока (> 10000).")

def _validate_discount(self, errors: List[str]) -> None:

    if self.special_discount_percent is not None:

        if self.special_discount_percent < 0 or self.special_discount_percent > 100:

            errors.append("Знижка має бути від 0% до 100%.")

        elif self.special_discount_percent == 100 and self.price > 0:

            errors.append("Страва не може бути повністю безкоштовною через знижку 100%.")

def _validate_image_url(self, errors: List[str]) -> None:

    if self.image_url:

        import re # тимчасово залишаємо – буде виправлено в операції №3

        url_pattern = re.compile(r'^https?:/\S+$')

        if not url_pattern.match(self.image_url):

            errors.append("Посилання на зображення має бути коректним URL (http:// або https://).")
```

ЧОМУ:

- Кожен маленький метод відповідає за перевірку одного конкретного поля – дотримано SRP.
- Основний метод став коротким (5 рядків) і читається як інструкція.
- Код легше тестувати (можна окремо перевірити `_validate_price` тощо).
- Полегшено супровід: зміна правил валідації назви не зачіпає решту логіки.

Регресійне тестування пройдено, рисунок 4.10 підтверджує це.

```
✓ Tests passed: 12 of 12 tests – 9 ms

===== test session starts =====
collecting ... collected 12 items

test_menu_item.py::test_calculate_final_price_available_no_discount PASSED [ 8%]
test_menu_item.py::test_calculate_final_price_unavailable PASSED [ 16%]
test_menu_item.py::test_calculate_final_price_special_discount_25 PASSED [ 25%]
test_menu_item.py::test_calculate_final_price_special_discount_boundaries PASSED [ 33%]
test_menu_item.py::test_calculate_final_price_multiple_seasonal_discounts PASSED [ 41%]
test_menu_item.py::test_calculate_price_with_vat_standard PASSED [ 50%]
test_menu_item.py::test_calculate_price_with_vat_reduced PASSED [ 58%]
test_menu_item.py::test_calculate_price_with_vat_invalid_rate PASSED [ 66%]
test_menu_item.py::test_calculate_price_with_vat_price_boundaries PASSED [ 75%]
test_menu_item.py::test_validate_for_publish_valid PASSED [ 83%]
test_menu_item.py::test_validate_for_publish_name_boundaries PASSED [ 91%]
test_menu_item.py::test_validate_for_publish_price_boundaries PASSED [100%]

===== 12 passed in 0.12s =====
```

Рисунок 4.10 – Проходження тестів з третьої лабораторної

Операція №3. Перенесення імпорту на початок файлу

Проблема: Імпорт модуля `re` знаходився всередині методу `_validate_image_url` (раніше – всередині `validate_for_publish`). Це сповільнює код при кожному виклику та порушує PEP 8.

БУЛО: `def _validate_image_url(self, errors: List[str]) -> None:`

```
    if self.image_url:

        import re

        url_pattern = re.compile(r'^https?:/\S+$')

        # ...
```

СТАЛО: `import re`

```

from typing import List, Optional, Callable

from decimal import Decimal, ROUND_HALF_UP

# ... решта імпортів

class MenuItem:

    # ...

    def _validate_image_url(self, errors: List[str]) -> None:

        if self.image_url:

            url_pattern = re.compile(r'^https?://S+$')

            if not url_pattern.match(self.image_url):

                errors.append("Посилання на зображення має бути коректним URL (http:// або https://).")

```

ЧОМУ:

- Відповідає стандарту PEP 8: всі імпорти мають бути на початку файлу.
- Усунуто накладні витрати на повторний імпорт при кожному виклику методу.
- Покращено читабельність: всі залежності видні зверху.

Регресійне тестування пройдено, рисунок 4.11 підтверджує це.

```

===== test session starts =====
collecting ... collected 12 items

test_menu_item.py::test_calculate_final_price_available_no_discount PASSED [ 8%]
test_menu_item.py::test_calculate_final_price_unavailable PASSED [ 16%]
test_menu_item.py::test_calculate_final_price_special_discount_25 PASSED [ 25%]
test_menu_item.py::test_calculate_final_price_special_discount_boundaries PASSED [ 33%]
test_menu_item.py::test_calculate_final_price_multiple_seasonal_discounts PASSED [ 41%]
test_menu_item.py::test_calculate_price_with_vat_standard PASSED [ 50%]
test_menu_item.py::test_calculate_price_with_vat_reduced PASSED [ 58%]
test_menu_item.py::test_calculate_price_with_vat_invalid_rate PASSED [ 66%]
test_menu_item.py::test_calculate_price_with_vat_price_boundaries PASSED [ 75%]
test_menu_item.py::test_validate_for_publish_valid PASSED [ 83%]
test_menu_item.py::test_validate_for_publish_name_boundaries PASSED [ 91%]
test_menu_item.py::test_validate_for_publish_price_boundaries PASSED [100%]

===== 12 passed in 0.16s =====

```

Рисунок 4.11 – Проходження тестів з третьої лабораторної

4.2.9 Повторний запуск SonarLint та літера. Фіксація кількості issues «після»

Кількість проблем SonarLint = 0, це показано на рисунку 4.12.



Рисунок 4.12 – Знаходження проблем через SonarLint

Кількість попереджень Ruff = 0, це показано на рисунку 4.13.

```
PS D:\Домашнє завдання\Інформатика\ООП\MenuQR> python -m ruff check "Unit Testing\menu_item.py"
All checks passed!
```

Рисунок 4.13 - Знаходження попереджень

4.2.10 Повторне обчислення метрик цикломатичної складності та індексу супроводжуваності

Було проведено повторне обчислення метрик цикломатичної складності та індексу супроводжуваності через radon, результати: цикломатична складність = 3.7, індекс супроводжуваності = 58 – дані підтверджено на рисунку 4.14.

```
PS D:\Домашнє завдання\Інформатика\ООП\MenuQR> python -m radon cc "Unit Testing\menu_item.py" -a
Unit Testing\menu_item.py
M 26:4 MenuItem.__init__ - B
M 50:4 MenuItem.calculate_final_price - B
M 124:4 MenuItem._validate_discount - B
C 19:0 MenuItem - A
M 143:4 MenuItem.set_special_discount - A
M 110:4 MenuItem._validate_name - A
M 116:4 MenuItem._validate_price - A
C 13:0 ValidationResult - A
M 131:4 MenuItem._validate_image_url - A
C 6:0 SeasonalDiscount - A
M 15:4 ValidationResult.__init__ - A
M 82:4 MenuItem.calculate_price_with_vat - A
M 8:4 SeasonalDiscount.__init__ - A
M 102:4 MenuItem.validate_for_publish - A
M 140:4 MenuItem.set_availability - A

15 blocks (classes, functions, methods) analyzed.
Average complexity: A (3.7333333333333334)
PS D:\Домашнє завдання\Інформатика\ООП\MenuQR> python -m radon mi "Unit Testing\menu_item.py" -s
Unit Testing\menu_item.py - A (58.21)
```

Рисунок 4.14 - Обчислення метрик цикломатичної складності та індексу супроводжуваності

4.2.11 Порівняння метрик до і після рефакторингу

Для наочнішого порівняння було зроблено таблицю 4.2, занесено дані.

Таблиця 4.2 – Порівняння метрик до і після рефакторингу

Метрика	До	Після
Цикломатична складність (max)	С (11-20) – метод <code>validate_for_publish</code>	В (6-10) – методи <code>__init__</code> , <code>calculate_final_price</code> , <code>_validate_discount</code>
Індекс супроводжуваності (середній)	A (60.87)	A (58.21)
SonarLint issues	0	0
Ruff warnings	0	0
Тести (pass / total)	12/12	12/12

Результат рефакторингу:

- Рефакторинг знизив максимальну та середню цикломатичну складність, покращивши структуру коду.
- Індекс підтримуваності залишився у високій категорії A.
- Усі тести успішно пройдено, поведінка програми не змінена.

Відрефакторингований код було оновлено та запущено на гітхаб:

https://github.com/mk1234717/MenuQR/blob/KuprinMatvii/Unit%20Testing/menu_item.py.

ВИСНОВКИ

Отримано навички Code Review через Pull Requests, виявлення code smells та рефакторингу з перевіркою тестами. Виконано 3 рефакторинг-операції, цикломатична складність знизилась з С до В, середня з 4.91 до 3.73, всі 12 тестів пройдено. Найцінніший досвід: чужий погляд допомагає знайти приховані проблеми, конструктивні коментарі прискорюють виправлення, регресійне тестування гарантує безпеку змін.